TM12 HELMET INSPECTION AUTOMATION SYSTEM

Technical Report

Communication Protocols · Vision Pipeline 1 (Sphere Arc) · Vision Pipeline 2 (Ellipse Orbit + Marking)

Politecnico di Milano — Machine Learning for Mechanical Systems — Semester 4

1. Introduction

This report documents the technical implementation of an automated quality inspection system for Nolan N120 motorcycle helmets, built around an Omron TM12 six-degree-of-freedom collaborative robot arm. A synthetic circular defect — represented by a green adhesive sticker placed on the helmet surface — must be detected, localised in three dimensions, and physically marked with a pen. The end effector is a custom box-shaped unit whose faces carry a ZED Mini stereo camera, a marker pen, and visor actuation tools.

Two independent vision pipelines were developed in parallel. Pipeline 1 (Hazik) uses a sphere arc scan strategy with a single-file monolithic structure and delivers validated 3D localisation without marking integration. Pipeline 2 (Kai) uses a horizontal ellipse orbit and a fully modular multi-file architecture that completes the end-to-end loop including physical marking. Both pipelines share the same robot communication infrastructure.

This report covers three topics in full technical depth: the three-channel robot communication protocol stack used by all pipelines; Pipeline 1 in detail; and Pipeline 2 in equivalent detail including its detection logic, transform calibration, multi-view fusion, and complete marking system.

2. Robot Communication Protocols

All pipelines communicate with the TM12 through three simultaneous TCP connections, each serving a distinct purpose. The robot IP is 192.168.1.3 in the lab configuration.

2.1 Architecture Overview

Channel	Protocol	Port	Direction	Primary purpose
Modbus TCP	Modbus over TCP	502	Read-only	Live TCP pose and joint angles — read before every camera capture
TMSCT	TM Script TCP	5890	Write + ACK	Motion commands sent to the TMFlow Listen Node
TMSVR	TM Ethernet Slave	5891	Read/Write	Named variable table — used to confirm Listen Node readiness

2.2 Modbus TCP — State Readback

Modbus TCP reads the robot's live kinematic state at any time without interrupting motion. The Python pymodbus client reads 16-bit holding registers and decodes each float as two consecutive registers in IEEE 754 single-precision big-endian format.

Registers	Parameter	Units
7013 – 7024	Joint angles J1 – J6	Degrees
7025 – 7036	TCP pose [X, Y, Z, Rx, Ry, Rz]	mm / degrees

The tcp_coord property reads registers 7025–7036 and returns [X, Y, Z, Rx, Ry, Rz]. Both pipelines call this immediately before every camera frame capture to obtain the current T_base_flange for the coordinate transform chain. If Modbus returns all-zeros, Modbus TCP is not enabled in TMFlow — it must be explicitly enabled under TMFlow Configuration → Ethernet → Modbus TCP.

2.3 TMSCT — Listen Node Control Channel

TMSCT on port 5890 is the write channel for all robot motion. It connects to the TMFlow Listen Node, a special state in the TMFlow program that suspends visual programming execution and accepts external TM Script commands. The Listen Node must be actively running before port 5890 opens — a failed connection almost always means the TMFlow program has not been started.

2.3.1 Packet Format

Every TMSCT transmission is a single ASCII frame terminated with CRLF. The id counter auto-increments per message and the robot uses it for duplicate and ordering detection.

```
$TMSCT,<length>,<id>,<command_data>,*<checksum>\r\n

  $TMSCT   — literal header
  length   — decimal byte count of the data portion only
  id       — integer counter, incremented on every send
  data     — one or more TM Script commands, semicolon-separated
  checksum — XOR of all bytes between $ and * (exclusive)
```

The id counter is a critical fragile point: if the robot rejects a packet for any reason, the sender's counter advances but the robot's expected counter does not. All subsequent packets fail silently — the robot does not move and no error is reported unless the ACK response is explicitly parsed. This was the confirmed root cause of several silent motion failures during development.

2.3.2 Motion Command Types and Synchronisation

Command	TM Script form	Interpolation	Notes
PTP	PTP("CPP", pose, speed, ...)	Joint-space	Used for all non-contact moves; robot selects joint path
LINE	Line("CPP", pose, speed, ...)	Cartesian linear	Required for pen touchdown and retract; kinematic infeasibility risk on some poses
JMOVE	JMOVE(joint_array)	Joint-space direct	Direct joint-angle move; used for J6 flip manoeuvres only
QueueTag	QueueTag(n)	—	Synchronisation marker; signals the Python side when reached
ScriptExit	ScriptExit()	—	Exits the script block, returns Listen Node to idle

After any motion command the pipeline sends a QueueTag and then calls wait_queue_tag(), which blocks until the robot signals the matching tag. Without this synchronisation, a second command can be dispatched while the first motion is still executing, corrupting the trajectory. LINE commands were found to trigger ERROR;1 (kinematic infeasibility) on several visor trajectory waypoints and were replaced with PTP for all visor motion.

> **Critical constraint:** ChangeBase and ChangeTCP commands must never be sent via TMSCT. They cause the robot to exit the Listen Node unpredictably, severing the control connection. All coordinate frame arithmetic is handled in Python via pure list offsets.

2.4 TMSVR — Ethernet Slave Variable Channel

TMSVR on port 5891 provides named read/write access to the robot's Ethernet table. In the inspection pipelines it is used primarily at startup to confirm Listen Node readiness. The ethernet_table class in tm_packet.py models the table as a Python dictionary, serialising to and from the $TMSVR wire format which mirrors the TMSCT framing convention.

2.5 Software Stack

Module	Role
tm_packet.py — TMPacket (ABC)	Base class: socket lifecycle, packet framing, XOR checksum, send/receive loop
tm_packet.py — TMSCT	Sends TM Script command strings to Listen Node; parses ACK/ERROR responses
tm_packet.py — TMSVR	Reads/writes named variables; wraps Ethernet table key-value model
techman.py — TM_Robot	Unified interface: wraps Modbus + TMSCT + TMSVR; exposes ptp(), line(), tcp_coord, joints, wait_queue_tag()
tm_motion_functions_V1_80.py	Translates Python motion calls (ptp, line, jmove, queue_tag) to TM Script strings; bugs fixed — see §6

3. Vision Pipeline 1 — Sphere Arc Scan (Hazik)

Pipeline 1 is a single-file detection system (helmet_pipeline_attempt_3.py) that commands the robot along hemisphere arc positions surrounding the helmet, captures ZED depth frames at each position, detects the green sticker, transforms detections into the robot base frame, and fuses them into a single 3D estimate. Marking was not integrated — the pipeline validates detection and localisation only.

3.1 Scan Geometry: Hemisphere Arcs

Rather than orbiting on a flat horizontal plane, Pipeline 1 sweeps four arcs on the surface of a sphere centred on the estimated helmet centre. Each arc occupies a distinct vertical half-plane: XZ+ (front), YZ+ (right), XZ− (rear), YZ− (left). Each arc starts at the safe home position directly above the helmet and sweeps inward, visiting progressively lower viewing elevations as the arm moves along the arc. This naturally varies the camera angle relative to the helmet surface at each stop, which improves depth quality for off-apex detections compared to a fixed-elevation orbit.

3.1.1 Arc Length Parameterisation

The primary control parameters are arc lengths in millimetres (XZ_ARC_LENGTH_MM, YZ_ARC_LENGTH_MM), not angles. The angular step and number of waypoints are derived from these lengths divided by the sphere radius:

```
theta     = arc_length_mm / sphere_radius_mm    # radians
n_steps   = max(1, round(arc_length_mm / step_arc_mm))
d_theta   = theta / n_steps
```

This keeps the physical camera travel per step consistent regardless of sphere size and makes the scan coverage predictable in physical units rather than angular ones.

3.1.2 Sphere Centre Computation and the 173 mm Bug

The sphere centre is the estimated 3D helmet centre location in the robot base frame. A critical subtlety: it must be computed from the camera position at safe home, not from the flange position. At the home orientation (Rx = −90°, Ry = 0°, Rz = 0°), the ZED camera sits approximately 173 mm ahead of the flange along the world Y axis. Using the flange XY as the sphere centre displaces every arc waypoint by 173 mm, causing the robot to orbit a point 173 mm off the helmet centre — an asymmetric scan with most positions missing the helmet. The correct implementation reads the camera position via get_camera_pose_base() before establishing the sphere:

```
# Correct: sphere centre from camera, not flange
cam_home  = get_camera_pose_base(robot, R_FC, t_FC)
sphere_ctr = [cam_home[0], cam_home[1], cam_home[2] - SPHERE_RADIUS_MM]

# Wrong (early version):
# sphere_ctr = [flange[0], flange[1], flange[2] - R]  ← 173 mm Y error
```

3.1.3 Arc Waypoint and Orientation Computation

Each waypoint position on a given arc is computed geometrically. For the XZ+ arc (sweeping toward the front of the helmet), up_dir = [0,0,1] and sweep_dir = [1,0,0]:

```
theta_k   = k * d_theta
cam_pos   = sphere_ctr + R*(cos(theta_k)*up_dir + sin(theta_k)*sweep_dir)
flange_pos = cam_pos - R_base_flange @ t_camera_offset
# t_camera_offset = [4.7, 21.15, 172.85] mm in flange frame
```

The TCP orientation at each waypoint is solved so the camera optical axis always points toward the sphere centre — a look-at solve that keeps the helmet centred in the field of view at every step. In the TM12's ZYX Euler convention (R = Rz · Ry · Rx), this requires solving Rx and Ry while holding Rz fixed.

3.2 Joint Configuration Guard

The TM12's IK has multiple solutions for any Cartesian pose. The wrist can flip between two configurations by simultaneously applying J4 ± 180°, negating J5, and applying J6 ± 180°. The controller chooses silently based on the current joint state, potentially causing large unexpected arm motions. Pipeline 1 guards against this in two ways. First, a configuration check verifies J5 and J6 are within tolerance of the known good scan home values before beginning the scan. Second, when a J6 wind-up is detected, the fix is a JMOVE J6 ± 180° (rotating J6 in place in joint space, no TCP motion) followed by a re-PTP to the same Cartesian target to lock the desired configuration.

Joint	Expected (good config)	IK flip value
J1	−94.05°	—
J2	23.83°	—
J3	−136.46°	—
J4	−67.25°	~+112.75°
J5	85.80°	~−85.80° (negated)
J6	−90.86°	~+89.14°

3.3 Per-Frame Detection Pipeline

At each arc position the pipeline captures for CAPTURE_SECONDS (typically 3 s at 30 fps). Each frame passes through five stages.

3.3.1 CLAHE Contrast Enhancement

The L channel of the LAB colour space is histogram-equalised with CLAHE (clip limit 2.0, 8×8 tile grid) before HSV conversion. This locally normalises contrast across the image and makes the green detection robust to the non-uniform fluorescent lighting in the lab without shifting the hue channel that the threshold depends on.

```
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
cl  = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
lab[:,:,0] = cl.apply(lab[:,:,0])   # L channel only
bgr = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
```

3.3.2 HSV Threshold

```
LOWER_GREEN = np.array([30, 40, 40], dtype=np.uint8)
UPPER_GREEN = np.array([95, 255, 255], dtype=np.uint8)
mask = cv2.inRange(hsv, LOWER_GREEN, UPPER_GREEN)
```

3.3.3 Morphological Cleanup

A morphological open (erosion → dilation) followed by a close (dilation → erosion) removes single-pixel noise and fills small holes caused by specular highlights or low-contrast sticker regions.

3.3.4 Contour Filter

All contours are evaluated by area (MIN_GREEN_AREA to MAX_GREEN_AREA) and circularity ($4\pi \times \text{Area} / \text{Perimeter}^2$, threshold 0.42). The circularity threshold permits shapes up to a ~4:1 ellipse aspect ratio. This is not a binding constraint in practice: a circular sticker viewed 20° off-perpendicular projects at circularity ≈ 0.97. The threshold primarily rejects elongated noise and background objects.

3.3.5 Contour-Mask Depth Extraction

For the best candidate contour, a filled pixel mask is drawn (cv2.drawContours, thickness = −1). Only pixels inside this filled contour are queried against the ZED point cloud. This is a deliberate design choice that avoids an 18-pixel fixed-radius sampling bug found in an earlier implementation, where the circular sampling patch extended outside the sticker onto the curved helmet surface behind it. That background contamination caused a systematic 1.4–1.6× depth scale error, verified experimentally by moving the robot a known distance and observing that the reconstructed P_global shift was proportionally wrong.

```
cm = np.zeros_like(mask)
cv2.drawContours(cm, [best_contour], -1, 255, -1)   # -1 = filled
ys, xs = np.where(cm > 0)
pts = [cloud[y,x] for y,x in zip(ys,xs) if is_valid(cloud[y,x])]

# Robust depth: median then drop furthest 20%, take mean of inliers
med = np.median([p[2] for p in pts])
inliers = [p for p in pts if abs(p[2]-med) < tol]
P_camera = np.mean(inliers, axis=0)   # [X,Y,Z] metres, camera frame
```

The ZED operates in NEURAL depth mode (convolutional network refinement of stereo disparity) at HD720 / 30 fps. The runtime confidence threshold is set to 50 — a moderate value that rejects clearly invalid depth estimates without over-filtering at low-texture surface regions.

3.4 Coordinate Transform Chain

P_camera in metres is transformed to robot base frame in three steps:

```
P_camera (m)   — ZED output, camera optical frame
  × 1000
P_camera (mm)
  R_FC @ P_camera + t_FC
P_flange (mm)  — in flange frame
  R_base_flange @ P_flange + t_flange_base   (from Modbus tcp_coord)
P_global (mm)  — robot base frame
```

In implementation this is collapsed into a single call: point_camera_to_base(tcp, P_camera_mm). The tcp six-vector [X,Y,Z,Rx,Ry,Rz] from Modbus is converted to a 4×4 homogeneous matrix using ZYX Euler convention (R = Rz·Ry·Rx), matching the TM12's native orientation representation.

3.4.1 R_FC Calibration

R_FC (3×3) maps from the ZED camera frame to the flange frame. It was derived first from SolidWorks measurements of the camera mounting geometry inside the end effector box, then refined empirically through a two-view consistency check: the robot was positioned at two distinct

poses both observing the same static sticker. R_FC was adjusted iteratively to minimise the distance between the two reconstructed P_global estimates, reducing the spread from an initial 156 mm to 46 mm. The R_FC[:,2] column (camera Z axis) must point in the direction the camera looks in the flange frame — when it was wrong in an early iteration, the camera depth axis mapped to world +Z (upward), giving Z estimates near 1050 mm (shoulder height) instead of the correct ~450 mm.

The camera translation from the flange face (t_FC = [4.7, 21.15, 172.85] mm in the flange frame) was measured from the SolidWorks end effector assembly.

3.5 Multi-View Fusion

Detections from all frames across all arc positions are pooled. A two-stage rejection filter reduces noise. Stage one discards any point further than CONSISTENCY_TOLERANCE_MM from the 3D median of all candidates. Stage two applies a pairwise consistency check: a point is kept only if at least MIN_VALID_GLOBAL_POINTS other points lie within CONSISTENCY_TOLERANCE_MM of it. The final estimate is the mean of surviving inliers. All fusion logic is implemented inline inside the single pipeline file rather than as a separate module.

3.6 Run Modes

Mode	Behaviour
full_auto	All four arcs run without pause
arc_by_arc	Pauses after each arc with per-arc statistics
step_by_step	Pauses before every waypoint; operator can abort or modify before execution
pick_arcs	Operator selects a subset of the four arcs to execute

4. Vision Pipeline 2 — Ellipse Orbit + Marking (Kai)

Pipeline 2 is the complete end-to-end system: scan, detect, fuse, and physically mark. It is structured as a modular multi-file pipeline with one file per responsibility, orchestrated by main_script.py. Unlike Pipeline 1, the architecture separates concerns cleanly — detection logic lives in vision_zed.py, trajectory and marking computation in robot_motion.py, stability judgement in decision.py, and tip compensation in marker_compensation.py. This section documents each module in full.

4.1 Top-Level Orchestration — main_script.py

main_script.py is the entry point. It owns the execution sequence and delegates every sub-task to the specialist modules below. The flow is:

```
1. Connect TM_Robot (Modbus + Listen Node) and ZedCamera.
2. [Optional] preview_camera_until_confirm() — live feed until operator
   confirms the helmet is correctly positioned in the camera frame.
3. generate_ellipsoid_slice_ellipse_scan_poses() — compute the 7-point
   ellipse orbit and per-point TCP orientations.
4. For each of the 7 scan poses:
```

```
      a. move_ptp(robot, pose) — robot moves to scan position
      b. Wait SETTLE_TIME seconds for vibration to damp out
      c. capture_defect_position_for_seconds(zed, detector, CAPTURE_SECONDS)
         — grab frames, run GreenDefectDetector, collect raw camera-frame
           detections
      d. build_runtime_camera_transform(robot) → T_base_camera
      e. camera_point_to_global(pt, T) for each detection
      f. [optional] ellipsoid boundary filter on each global point
      g. Accumulate valid P_global points
5. judge_global_points_stability(all_points) → (stable, P_final_mm)
6. If stable: build_radial_marking_plan(robot, P_final_mm)
7. If RUN_FINAL_MARKING=True: send_motion_and_wait(robot, plan)
```

During all robot moves, a ZED frame is grabbed and displayed with the ellipsoid boundary projected on the image, giving the operator a live view of the arm's position relative to the helmet throughout the scan.

4.2 Camera and Sensor — vision_zed.py

vision_zed.py is the ZED SDK wrapper and the primary detection engine. It contains both the camera infrastructure and the per-frame sticker detection logic.

4.2.1 ZedCamera Class

ZedCamera opens the ZED Mini with the configured resolution (HD720), frame rate (15 fps in Pipeline 2's config), and NEURAL depth mode. It calls sl.Camera.enable_positional_tracking() at init, which activates the ZED's on-board visual odometry. The primary methods exposed are grab() — which returns a paired (bgr_image, point_cloud) — and get_left_camera_intrinsics(), which returns the camera matrix for optional projection operations.

4.2.2 GreenDefectDetector and Per-Frame Detection

GreenDefectDetector is instantiated with the HSV bounds, area limits, and circularity threshold from config.py. Its detect() method runs the per-frame pipeline. Compared to Pipeline 1, Pipeline 2 adds two additional rejection stages after the HSV mask: RGB ratio checks and a depth clustering step.

The full per-frame pipeline is:

```
1. Convert BGR → HSV
2. cv2.inRange(hsv, LOWER_GREEN, UPPER_GREEN) → binary mask
3. RGB ratio check on the raw image at masked pixels:
      keep only pixels where:
        G / R >= MIN_GREEN_OVER_RED   (e.g. 1.2)
        G / B >= MIN_GREEN_OVER_BLUE  (e.g. 1.1)
   This rejects false positives from white or yellow objects that pass
   a loose HSV green range due to lighting shifts.
4. Morphological open + close (noise cleanup, same as Pipeline 1)
5. Contour filter: area in [MIN_GREEN_AREA, MAX_GREEN_AREA] and
   circularity = 4π·Area/Perimeter² >= MIN_CIRCULARITY
6. For best contour: back-project to 3D via ZED point cloud.
   Depth clustering step — bin the per-pixel depth values from the
   contour region into a histogram; take the dominant cluster's
   median rather than the raw per-pixel median. This reduces
   sensitivity to depth noise from reflective sticker edges.
7. [Optional] ellipsoid boundary check on the 3D point.
8. Return DefectDetectionResult(centroid_px, area_px, point_camera_m)
```

The RGB ratio checks in step 3 are a meaningful addition over Pipeline 1. They address a specific failure mode in the lab: the overhead fluorescent lights can cause white surfaces to exhibit a slight

warm-green cast in the HSV image, passing the hue threshold. The ratio check ensures the raw green channel intensity genuinely exceeds red and blue by a margin, which is true for the sticker but not for misidentified white surfaces.

4.2.3 Data Classes

Class	Fields	Purpose
DefectDetectionResult	centroid_px, area_px, point_camera_m	Single-frame detection output from GreenDefectDetector.detect()
FrameRecord	detection_result, camera_pose_snapshot, timestamp	Stores one frame's detection together with the TCP pose at that instant
CameraEstimate	point_camera_m, std_mm, quality_flags	Per-view fused estimate after analyze_frame_records() runs over a set of FrameRecords

4.2.4 capture_defect_position_for_seconds

This function drives the per-pose capture loop. It grabs frames from ZedCamera for the configured CAPTURE_SECONDS_PER_VIEW duration, runs GreenDefectDetector.detect() on each, and accumulates valid FrameRecords. After capture, it calls analyze_frame_records() to produce a CameraEstimate for the view. The function returns a list of global-frame 3D points (after transform) to the calling loop in main_script.py.

4.2.5 analyze_frame_records — Per-View Robust Statistics

analyze_frame_records() receives all FrameRecords from one scan position and computes a robust fused estimate in three passes: first the median centroid and depth across all frames, then trimming the worst 20% of frames by distance from the median, then the mean of the remaining inliers. It also computes std_mm (standard deviation of the per-frame 3D points) and sets quality flags that caller code can inspect to decide whether the view's estimate is trustworthy.

4.2.6 preview_camera_until_confirm

At startup (if enabled), this function enters an interactive loop showing the live ZED feed with the ellipsoid boundary projected on the image. The operator presses a key to confirm the helmet is correctly framed and the scan can begin. This provides a visual sanity check before any robot motion.

4.3 Scan Trajectory and Orientation — robot_motion.py

robot_motion.py generates the scan trajectory, implements the orientation solve for each scan pose, provides workspace safety checks, and computes the full marking motion plan. It is one of the two most complex modules in Pipeline 2 (the other being decision.py).

4.3.1 generate_ellipsoid_slice_ellipse_scan_poses

This function generates the 7-point horizontal ellipse orbit. All scan points are at a fixed Z height (the scan elevation), varying only in XY. Points are distributed evenly around the ellipse by angle:

```
for i in range(N_SCAN_POINTS):           # N=7
    angle = 2*pi * i / N_SCAN_POINTS
    x = helmet_centre[0] + ELLIPSE_RADIUS_X * cos(angle)
```

```
    y = helmet_centre[1] + ELLIPSE_RADIUS_Y * sin(angle)
    z = SCAN_HEIGHT_Z
    orientation = solve_rx_ry_for_local_axis(
        target_dir = helmet_centre - [x,y,z],   # camera looks toward helmet
        rz_fixed   = RZ_FIXED                   # keeps camera upright
    )
    poses.append([x, y, z, *orientation])
```

The orientation solve is critical: without it, the camera would point in a fixed direction as it orbits, and the helmet would drift in and out of the field of view as the robot circles around it. With the solve, every scan point has the camera optical axis aimed at the helmet centre regardless of the robot's position on the ellipse.

The ellipse shape (rather than a circle) accounts for the fact that the robot's reachable workspace is not uniform in all directions from the helmet position — an ellipse can be tuned to keep all 7 poses within a comfortable operating range.

4.3.2 solve_rx_ry_for_local_axis — Orientation IK Helper

This function solves for the Rx and Ry Euler angles (in ZYX convention) given a desired world-frame direction for the tool's local axis (camera Z for scan poses, marker axis for marking poses), with Rz held fixed. The derivation uses the fact that in ZYX convention, the rotation matrix columns are the world-frame directions of the tool's X, Y, Z axes. Setting column 2 equal to the normalised target direction and extracting Rx, Ry via atan2 gives the unique solution for the given Rz.

```
def solve_rx_ry_for_local_axis(target_dir, rz_fixed):
    target_dir /= np.linalg.norm(target_dir)
    Rz = rot_z(rz_fixed)
    # Desired col2 of R = Rz @ Ry @ Rx must equal target_dir
    # Decompose: ry = asin(-target_dir[2]) after undoing Rz
    d = Rz.T @ target_dir
    ry = np.arcsin(-d[2])
    rx = np.arctan2(d[1], d[0])   # adjusted for ZYX convention
    return [np.degrees(rx), np.degrees(ry), np.degrees(rz_fixed)]
```

4.3.3 Workspace Safety Checks

move_ptp() and move_line() in robot_motion.py are thin wrappers around TM_Robot.ptp/line that first call is_pose_safe(). This function checks the target position against configurable SAFE_X_RANGE, SAFE_Y_RANGE, SAFE_Z_RANGE bounds in config.py. If a pose falls outside the bounds, ensure_pose_inside_safe_workspace() clamps it and logs a warning before the move is sent. This prevents the scan trajectory or marking plan from commanding the robot into a clamp, the table surface, or the physical limits of the workspace.

4.4 Camera Transform — coordinate_transform.py

Pipeline 2 uses CAMERA_RELATIVE_POSE_TCP = [26.9, 14.1, 187.0] mm as the camera-to-TCP translation offset, with CAMERA_ROTATION_MATRIX_TCP as the rotation override. Two key functions handle the transform:

4.4.1 build_camera_to_base_matrix — build_runtime_camera_transform

At each scan position, the current TCP pose is read from Modbus and converted to T_base_tcp (4×4 homogeneous matrix, ZYX Euler). The camera-to-base matrix is then:

```
T_tcp_cam  = pose6d_to_transform(CAMERA_RELATIVE_POSE_TCP)
```

```
if CAMERA_ROTATION_MATRIX_TCP is not None:
    T_tcp_cam[:3,:3] = CAMERA_ROTATION_MATRIX_TCP   # calibrated override

T_base_tcp = pose6d_to_transform(robot.tcp_coord)  # live from Modbus
T_base_cam = T_base_tcp @ T_tcp_cam
```

4.4.2 6-View Rotation Matrix Calibration

CAMERA_ROTATION_MATRIX_TCP was derived from an empirical calibration run on 09-06-2026. The robot was placed at 6 distinct poses all pointing at the same known fixed sticker, and the rotation matrix was optimised to minimise the total spread across all 6 P_global estimates. Six views give a significantly more overdetermined system than Pipeline 1's 2-view procedure, producing a matrix more stable across a wider range of approach angles. The matrix is hardcoded into config.py.

4.4.3 camera_point_to_global

With T_base_cam available, a camera-frame 3D point is transformed to the base frame as a standard homogeneous product:

```
def camera_point_to_global(p_cam_m, T_base_cam):
    p_cam_mm  = np.array([*p_cam_m, 1.0]) * 1000   # metres → mm, homogeneous
    p_cam_mm[3] = 1.0
    p_global_mm = T_base_cam @ p_cam_mm
    return p_global_mm[:3]   # [X, Y, Z] in mm, robot base frame
```

4.5 Ellipsoid Boundary Filter — ellipsoid_boundary.py

ellipsoid_boundary.py implements a 3D spatial filter that rejects candidate P_global points falling outside a configurable ellipsoidal volume around the expected helmet. The ellipsoid is defined by dimensions 270 × 400 × 284 mm (half-axes in X, Y, Z of the robot base frame) centred on HELMET_CENTER_GLOBAL_MM.

The point-in-ellipsoid test is the standard normalised quadratic form: a point p is inside if $(\Delta x/a)^2 + (\Delta y/b)^2 + (\Delta z/c)^2 \leq 1$, where Δ is the offset from the ellipsoid centre and a, b, c are the semi-axes. The function draw_ellipsoid_projection_on_image() renders the boundary as an overlay on the live ZED preview, giving the operator visual confirmation that the helmet is within the expected region before scanning begins. The filter is disabled by default (ENABLE_ELLIPSOID_BOUNDARY = False) because at steep scan angles near the helmet rim it occasionally rejects valid detections.

4.6 Multi-View Stability Judge — decision.py

decision.py receives all accumulated P_global points from the 7 scan positions and decides whether they are consistent enough to proceed with marking. The module implements its logic as a chain of distinct functions rather than inline code, making each step individually testable.

4.6.1 reject_global_outliers

Computes the 3D median of all input points and removes any point whose distance from the median exceeds GLOBAL_OUTLIER_DISTANCE_MM. This eliminates gross outliers from, for example, a false positive detection at one scan position that picked up a background green object rather than the sticker.

4.6.2 robust_fuse_global_points

After outlier rejection, the remaining points are fused: compute median, trim the worst 20% by distance from median, take the mean of the survivors. This triple-pass approach is robust to the moderate noise distribution typical of stereo depth measurements at the scan distances used.

4.6.3 judge_global_points_stability

This is the main entry point for main_script.py. It calls reject_global_outliers, then checks that all surviving points are within CONSISTENCY_TOLERANCE_MM of each other (pairwise_max_distance_mm). If they are, and if at least MIN_VALID_GLOBAL_POINTS remain, it calls robust_fuse_global_points and returns (True, P_final_mm, reason_string). Otherwise it returns (False, None, reason_string) and marking is not attempted. The reason string gives the caller an auditable explanation of why the judgement was made.

```
def judge_global_points_stability(points):
    inliers = reject_global_outliers(points, GLOBAL_OUTLIER_DISTANCE_MM)
    if len(inliers) < MIN_VALID_GLOBAL_POINTS:
        return False, None, f'Only {len(inliers)} points after outlier rejection'
    max_d = pairwise_max_distance_mm(inliers)
    if max_d > CONSISTENCY_TOLERANCE_MM:
        return False, None, f'Max pairwise distance {max_d:.1f} mm > tolerance'
    P_final = robust_fuse_global_points(inliers)
    return True, P_final, f'{len(inliers)} consistent points, max_d={max_d:.1f} mm'
```

4.7 Marking Plan — robot_motion.build_radial_marking_plan

Once judge_global_points_stability returns True with a confirmed P_final_mm, build_radial_marking_plan() computes the full motion plan for contacting the defect with the marker pen. 'Radial' describes the approach direction: the robot approaches along the vector from the helmet centre to the defect. Since the helmet is approximately spherical, approaching radially aligns the marker axis approximately normal to the surface at the defect location without requiring an explicit surface normal estimate.

4.7.1 Step 1 — Safety Approach Pose

The nearest of the 7 ellipse scan positions to the defect (in angular terms) is selected as the initial approach. The robot moves to this pose first, entering the marking corridor from a known, clear position rather than from an arbitrary robot state that might sweep the arm through clamp geometry.

4.7.2 Step 2 — Solve TCP Orientation for Marker Axis

The same solve_rx_ry_for_local_axis() function used for scan poses is called again, this time with the desired direction being the inward radial from the defect to the helmet centre — the direction the marker axis should point so it contacts the surface perpendicularly. Rz is held at the value from the nearest scan pose for continuity.

4.7.3 Step 3 — Tip Offset Compensation

The TCP origin is at the flange face. The marker pen tip is physically offset from it by MARKER_RELATIVE_POSE_TCP = [−24, 135, 149] mm in the flange frame. If the robot moves the TCP to P_final_mm, the tip lands 135 mm above and 149 mm behind the defect — a complete miss. The compensation requires the offset to be rotated into the base frame at the solved orientation, then subtracted from the target:

```
# compensate_marker_point_global() in marker_compensation.py:
```

```
T_base_tcp    = pose6d_to_transform(current_tcp)   # 4×4
T_tcp_marker  = pose6d_to_transform(MARKER_RELATIVE_POSE_TCP)
T_base_marker = T_base_tcp @ T_tcp_marker

offset_base   = T_base_marker[:3,3] - T_base_tcp[:3,3]   # mm in world frame
tcp_target_xy = P_final_mm[:2] - offset_base[:2]

# Z: calibrated constant — NOT from detection
tcp_target_z  = MARKING_TOUCH_TCP_Z_MM + MARKING_TOUCH_CLEARANCE_MM
tcp_target    = [tcp_target_xy[0], tcp_target_xy[1], tcp_target_z, rx, ry, rz]
```

The orientation-first order is mandatory: offset_base is the tip offset rotated by R_base_tcp at the planned marking orientation. If the robot executed at a different orientation, R_base_tcp would differ and the tip would miss laterally. The plan solves orientation first, then computes the offset at that orientation, guaranteeing consistency.

4.8 Fixed-Z Strategy and Validity Domain

The marking touch Z uses a physically calibrated constant (MARKING_TOUCH_TCP_Z_MM) measured by jogging the robot until the pen tip physically touches the top of the helmet and recording the TCP Z. This is more reliable than the detected defect Z from the scan because ZED depth at the oblique ellipse scan angles has 5–15 mm of frame-to-frame noise on the reconstructed Z coordinate.

The strategy is only valid when the defect is close to the helmet apex. The surface drops away from the apex as a function of radial offset r on a sphere of radius R:

Radial offset r	$\Delta Z = R - \sqrt{(R^2 - r^2)}$ (R=130 mm)	Assessment
10 mm	0.4 mm	✓ Negligible
20 mm	1.6 mm	✓ Within compliance margin
35 mm	4.9 mm	✓ Borderline safe
50 mm	10.0 mm	⚠️ Geometric correction needed
80 mm	27.5 mm	✗ Pen misses surface
100 mm	46.9 mm	✗ Significant miss

The physical compliance failsafe is a spring-loaded adjustable sleeve on the end effector that holds the marker. The tip is set slightly longer than the nominal calibrated distance, so residual Z errors within ±2–3 mm are absorbed as controlled overtravel into the sleeve rather than breaking the tip. A geometric correction tier ($Z_{touch} = Z_{apex} - \Delta Z(r)$) using the spherical model would extend valid range to the full helmet sides without new calibration hardware, at the cost of approximately 20 additional lines of code.

4.9 Motion Execution — send_motion_and_wait

The marking motion executes as a five-step Cartesian sequence. Steps 3 and 5 use LINE (not PTP) so the pen descends and retracts in a straight Cartesian line, preventing lateral drift during contact.

Step	Motion type	Target	Purpose
1 — Safety pose	PTP	Nearest ellipse scan position	Enter marking corridor from known clear position

Step	Motion type	Target	Purpose
2 — Approach (above)	PTP	tcp_target with Z = touch_Z + CLEARANCE	Move to directly above defect at standoff height
3 — Touch descent	LINE	tcp_target with Z = touch_Z	Straight down to surface — LINE prevents lateral drift
4 — Dwell	wait	—	Hold for MARK_DWELL_SECONDS (0.3–0.5 s) for ink transfer
5 — Retract	LINE	tcp_target with Z = touch_Z + CLEARANCE	Straight up along same axis — prevents smearing

5. Pipeline Comparison

Both pipelines solve the same detection and localisation problem. Their design choices reflect different priorities: Pipeline 1 optimises for physical scan quality and step-by-step validation; Pipeline 2 optimises for end-to-end completion with modular, testable code.

Aspect	Pipeline 1 — Sphere Arc (Hazik)	Pipeline 2 — Ellipse Orbit (Kai)
Entry point	helmet_pipeline_attempt_3.py	main_script.py
Scan geometry	4 sphere arcs: XZ±, YZ± — varying elevation	7-point horizontal ellipse — fixed Z elevation
Scan points	~12 per arc × 4 arcs (configurable)	7 fixed points
Orientation solving	Look-at per arc step — camera toward sphere centre	Look-at per ellipse point — camera toward helmet centre
Camera transform	R_FC from SolidWorks + 2-view empirical calibration	CAMERA_RELATIVE_POSE_TCP + 6-view empirical rotation matrix
Detection extra step	None beyond HSV + contour	RGB ratio checks (G/R, G/B thresholds) + depth clustering
Depth extraction	Contour-mask only (fixed 18px bug avoided)	Depth histogram clustering within contour
Per-view fusion	analyze_frame_records inline	analyze_frame_records in vision_zed.py
Multi-view fusion	Inline consistency check	Separate decision.py (reject_global_outliers → pairwise → robust_fuse)
Ellipsoid filter	Disabled by default	Available via ENABLE_ELLIPSOID_BOUNDARY
Marking	Not integrated	Full build_radial_marking_plan + compensate_marker_point_global
Code structure	Monolithic single file	Modular: main_script + vision_zed + robot_motion + decision + marker_compensation

Aspect	Pipeline 1 — Sphere Arc (Hazik)	Pipeline 2 — Ellipse Orbit (Kai)
Operator control	Step-by-step mode with per-waypoint confirmation	Full-auto once started; live video feed during moves
Status	Detection ✓ Marking ✗	Full end-to-end ✓

6. Known Issues, Bugs Fixed, and Engineering Decisions

Issue / Decision	Detail	Resolution
ChangeBase / ChangeTCP prohibition	Sending either command via TMSCT causes the robot to exit the Listen Node, severing the control connection.	All frame arithmetic is pure Python list offsets. ORIGIN is a mutable list; all downstream code sees updates in place.
LINE commands — kinematic infeasibility (ERROR;1)	Several visor trajectory waypoints failed with ERROR;1 when sent as LINE commands.	All visor trajectory commands converted to PTP. LINE retained only for the marking touch/retract steps where Cartesian linearity is required.
MotionOptions class-level dict mutation	Original tm_motion_functions_V1_80.py used class-level dicts. ptp(blending=0) permanently mutated the shared default for all subsequent calls in the process.	Fixed with per-instance copy.copy() of option dicts in __init__.
@staticmethod on instance method	A method in tm_motion_functions that used self was decorated @staticmethod, causing TypeError at runtime.	Decorator removed.
Fixed 18px depth sampling — 1.4× scale error	An early depth extraction sampled a fixed 18px circle around the centroid, including curved helmet surface outside the sticker. This caused a confirmed ~1.4–1.6× depth scale error, verified by moving the robot a known distance and comparing P_global shift.	Replaced with contour-mask sampling in both pipelines. Verified with transform_validate.py test A.
Sphere centre from flange — 173 mm Y error (Pipeline 1)	Early version computed sphere centre from the flange XY. Camera sits 173 mm ahead of flange in world Y at home orientation.	Sphere centre now computed from get_camera_pose_base() which accounts for the camera-to-flange offset.
J6 wind-up / IK flip	Unchecked PTP commands to orientation-changing poses caused the controller to silently select an alternative IK configuration, producing large wrist flip motions.	J5/J6 guard check at scan home entry; JMOVE J6 ±180° in place followed by re-PTP to lock the desired configuration.
techman.py move_line	The move_line method in the	Corrected to call the line command

Issue / Decision	Detail	Resolution
called move_ptp internally	teammate's techman.py contained a copy-paste error and called move_ptp instead of the line command.	method.
Fixed-Z marking limited to helmet apex	MARKING_TOUCH_TCP_Z_MM is calibrated at the apex. ΔZ grows as $R-\sqrt{(R^2-r^2)}$; at r=100 mm the pen misses the surface by ~47 mm.	Documented as a known limitation. Tier-2 fix: sphere-model geometric correction requires ~20 lines of code and no new calibration.

7. Conclusion

The TM12 helmet inspection system is built on three simultaneous robot communication channels: Modbus TCP for continuous kinematic state readback, TMSCT for scripted motion control through the Listen Node, and TMSVR for variable table access. The prohibition on ChangeBase and ChangeTCP and the management of all coordinate arithmetic in Python keep the robot controllable and the connection stable throughout multi-minute scan cycles.

Pipeline 1 demonstrates that a sphere arc scan geometry with CLAHE-enhanced HSV detection and contour-mask depth extraction produces repeatable 3D localisation, with the 2-view R_FC calibration validated to 46 mm spread. Its step-by-step run mode makes it the appropriate tool for verifying each motion stage individually before committing to automated operation.

Pipeline 2 completes the full inspection loop. Its per-frame detection adds RGB ratio checks and depth clustering over Pipeline 1, its 6-view camera calibration is more overdetermined, its multi-view fusion is modularised and independently testable in decision.py, and its marking system implements the orientation-first tip-offset compensation correctly via compensate_marker_point_global. The fixed-Z marking strategy is reliable within ±35 mm of the helmet apex; a geometric sphere-model correction extends this to the full helmet surface at minimal implementation cost.

As of submission, Pipeline 1 has validated detection and localisation. Pipeline 2 has a validated detection and marking implementation; end-to-end physical marking on the helmet surface is the final integration step remaining.